Beschreibung Testframework

Author: Patrick Dressel
Datum: 12.12.2016

2. Inhaltsverzeichnis

3. Änderungsdokumentation **3**

4. Aufgabenstellung **3**

5. Begriffe **3**

6. Software **4**

6.1. Klassendiagramm *4*

6.2. Klassen *5*

6.2.1. CUnitTestLib 5

6.2.1.1. Übersicht 5

6.2.1.2. Schnittstelle 5

6.2.1.3. Methoden 6

6.2.1.4. Datentypem 6

6.2.2. CTestSuite 7

6.2.2.1. Übersicht 7

6.2.2.2. Schnittstelle 7

6.2.2.3. Methoden 7

6.2.2.4. Datentypen 8

6.2.3. CUnitTestBase 9

6.2.3.1. Beschreibung 9

6.2.3.2. Übersicht 9

6.2.3.3. Elemente 9

6.2.3.4. Schnittstelle 9

6.2.3.5. Methoden 10

6.2.3.6. Assert Methoden 11

6.2.3.7. Datentypen 13

6.2.4. CUnitTestScreen 14

6.2.4.1. Beschreibung 14

6.2.4.2. Übersicht 14

6.2.4.3. Elemente 14

6.2.4.4. Schnittstelle 14

6.2.4.5. Methoden 15

6.2.5. CScreenInterface 16

6.2.5.1. Übersicht 16

6.2.5.2. Schnittstelle 16

6.2.5.3. Methoden 16

6.3. Beschreibung des Beispiels *17*

6.3.1. Anwendungstest 17

6.3.2. Visualisierungstest 17

3. Änderungsdokumentation

Revision	Datum	Änderung	von
0.1	12.12.2016	Erstellung	Patrick Dressel

4. Aufgabenstellung

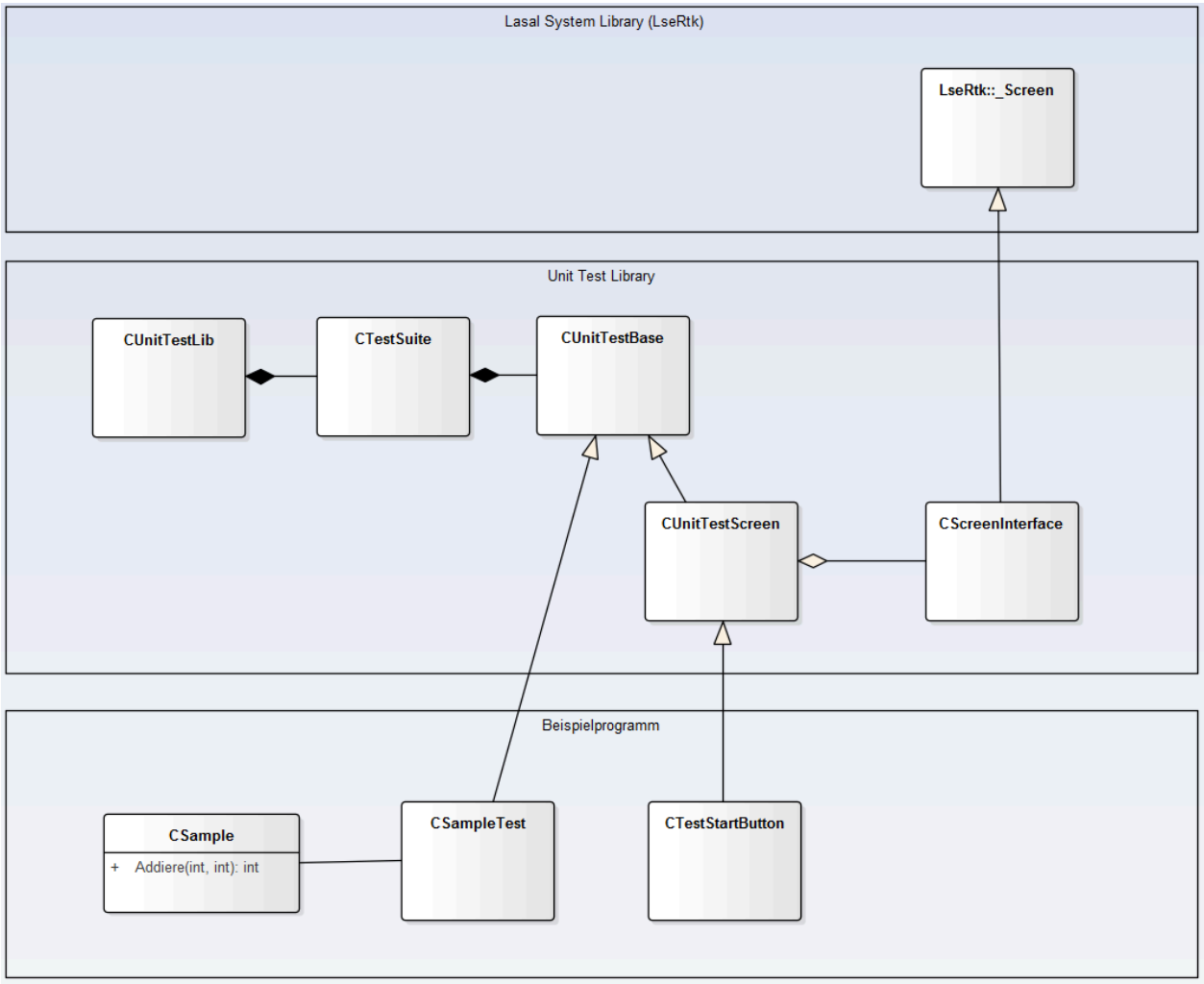
Für die Steuerung der Firma Sigmatek gibt es ein eigenes Entwicklungstool Lasal Class (Ablauf) und Lasal Screen (Visualisierung).
Um in der Umgebung Unittests zu realisieren soll eine Bibliothek mit allen notwendigen Klassen erstellt werden die einen automatisierten Testablauf möglich machen.

5. Begriffe

Begriff	Erklärung

6. Software

6.1. Klassendiagramm

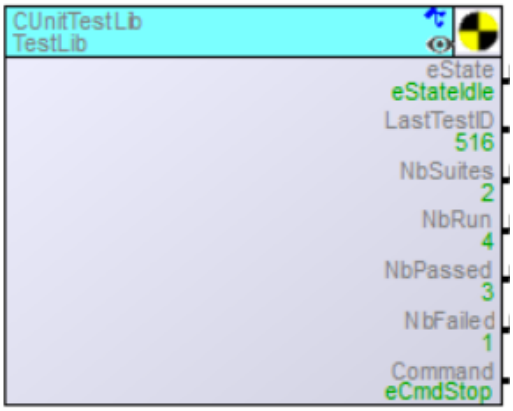


6.2. Klassen

6.2.1. CUnitTestLib

Diese Klasse stellt die Hauptverbindung zum Framework dar. Von hier aus werden die Tests gestartet und nach Ende der Tests die abschließende Auswertung erstellt. Die Instanz kann beliebig viele Instanzen von CTestSuite verwalten.

6.2.1.1. Übersicht



6.2.1.2. Schnittstelle

Client	Beschreibung
-	

Server	Beschreibung
eState	Status
LastTestID	Fortlaufender Zähler (RAM)
NbSuites	Anzahl registrierter Testsuites
NbRun	Gesamtzahl ausgeführter Tests (letzter Lauf)
NbPassed	Gesamtzahl erfolgreicher Tests (letzter Lauf)
NbFailed	Gesamtzahl fehlerhafter Tests (letzter Lauf)
Command	Befehle für Teststart *eCmdRunAll* *eCmdRunSelected*

6.2.1.3. Methoden

CyWork
Zyklischer Programmteil

registerSuite	
Neue Testsuite hinzufügen	
pSuite	Adresse der neuen Instanz
bRetcode	true = hinzugefügt false = nicht hinzugefügt

createProtocol
Diese Methode erstellt einen Bericht über den letzten Testlauf. Der Bericht enthält - Name Der

6.2.1.4. Datentypem

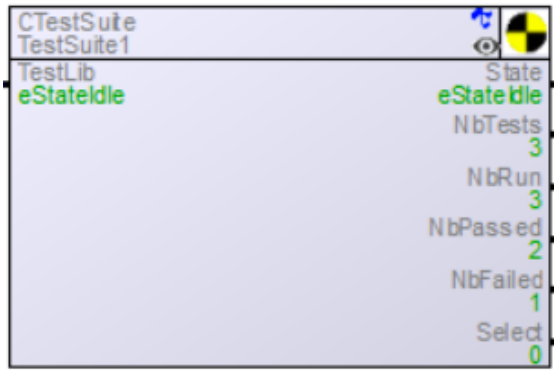
EState_t	
Status	
eStateIdle	
eStateExecute	
eStateNextSuite	
eStateError	
eStateCreateReport	

ECommand_t	
Kommando	
eCmdStop	
eCmdRunAll	
eCmdRunSelected	
eCmdReset	

6.2.2. CTestSuite

Die Klasse CTestSuite dient als Container für spezifische Tests. Hier können z.B. bestimmte Testarten zusammengefasst werden. Jede Testsuite kann beliebig viele Einzeltests besitzen.

6.2.2.1. Übersicht



6.2.2.2. Schnittstelle

Client	Beschreibung
TestLib	

Server	Beschreibung
State	Status der Instanz
NbTests	Anzahl registrierter Tests
NbRun	Anzahl ausgeführter Tests (letzter Lauf)
NbPassed	Anzahl erfolgreicher Tests (letzter Lauf)
NbFailed	Anzahl fehlerhafter Tests (letzter Lauf)
Select	Auswahl

6.2.2.3. Methoden

init
Initialisierung der Instanz

registerTest	
Test hinzufügen	
pSuite	Adresse der neuen Instanz
bRetcode	true = hinzugefügt false = nicht hinzugefügt

execute	
Mit der Funktion werden alle registrierten Tests ausgeführt	
eCommand	Befehl was ausgeführt werden soll *eCmdRunAll* *eCmdRunSelected*
-	

getStatistics	
Auslesen der Werte aus dem letzen Testlauf	
pVariable	Adresse auf Statistikstruktur
bRetcode	true = erfolgreich false = Fehler

6.2.2.4. Datentypen

SStatistic_t	
Statistikwerte	
udElapsedTime	Adresse auf Statistikstruktur
NbRun	Anzahl durchgeführter Tests
NbPassed	Anzahl erfolgreicher Tests
NbFailed	Anzahl fehlerhafter Tests

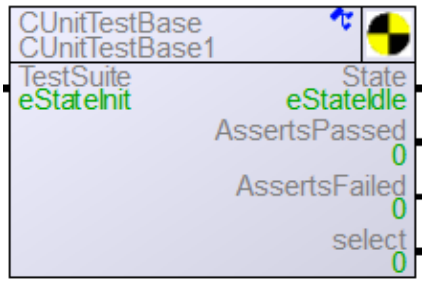
EState_t	
Status	
eStateIdle	
eStatePrepare	
eStateExecute	
eStateCleanUp	
eStateNextTest	
eStateError	

6.2.3. CUnitTestBase

6.2.3.1. Beschreibung

Diese Klasse dient als Basisklasse für den Test von Applikations-klassen und Funktionen. Der Anwender muss eine konkrete Testklasse davon ableiten und kann diese dann bei der Testsuite registrieren. In der abgeleiteten Testklasse kann der Anwender dann die virtuellen Methoden überladen.
Es muss mindestens *execute*(...) überladen werden sonst wird der Test auf FAILED gesetzt.

6.2.3.2. Übersicht



6.2.3.3. Elemente

Name	Beschreibung
m_strID	EIndeutige ID des Testfalls

6.2.3.4. Schnittstelle

Client	Beschreibung
TestSuite	Verbindung zur Testsuite Instanz

Server	Beschreibung
State	Aktueller Status des Tests
AssertsPassed	Anzahl erfolgreicher Asserts
AssertsFailed	Anzahle fehlerhafter Asserts
Select	Auswahl

6.2.3.5. Methoden

init
Initialisieren der Instanz

virtual prepare	
Die Funktion wird vor dem eigentlichen Test aufgerufen und dient dazu notwendige Vorbereitungen für den Test durchzuführen.	
bRetcode	true = Vorbereitung beendet false = Vorbereitung läuft

virtual execute	
Diese Funktion wird für den Test aufgerufen Über den Rückgabewert kann der Anwender mitteilen ob der Test beendet ist oder noch läuft.	
bRetcode	true = Test beendet false = Test läuft

virtual cleanup	
Die Funktion wird nach dem Test aufgerufen	
bRetcode	true = Cleanup beendet false = Cleanup läuft

prepareTest	
Die Funktion wird von der Testsuite aufgerufen.	
bRetcode	true = Vorbereitung beendet false = Vorbereitung läuft

executeTest	
Die Funktion wird von der Testsuite aufgerufen.	
bRetcode	true = Test beendet false = Test läuft

cleanupTest	
Die Funktion wird von der Testsuite aufgerufen.	
bRetcode	true = Cleanup beendet false = Cleanup läuft

6.2.3.6. Assert Methoden

Asserts bilden die Testbedingungen ab. Eine Assertion kann man auch als Behauptung betrachten. Im Falle der Unittests werden diese dazu verwendet um z.B. das Ergebnis von Funktionen zu prüfen. Zu allen Asserts gibt es die FATAL-Variante. Diese sorgt bei einem Fehler zum Abbruch der jeweiligen Testsuite.

assertTRUE		
Prüfung auf TRUE		
bValue	BOOL	Eingangswert

assertFALSE		
Prüfung auf FALSE		
bValue	BOOL	Eingangswert

assertEqual		
Prüfung auf Gleichheit (Integer)		
Actual	DINT	Istwert
Expected	DINT	Sollwert

assertNotEqual		
Prüfung auf Gleichheit (Integer)		
Actual	DINT	Istwert
Expected	DINT	Sollwert

assertPtrEqual		
Prüfung auf Gleichheit (Adresse)		
Actual	^VOID	Istwert
Expected	^VOID	Sollwert

assertPtrNotEqual		
Prüfung auf Gleichheit (Adresse)		
Actual	^VOID	Istwert
Expected	^VOID	Sollwert

assertPtrNULL		
Prüfung auf NULL		
Actual	^VOID	Istwert

assertPtrNotNULL		
Prüfung auf ungleich NULL		
Actual	^VOID	Istwert

assertStringEqual		
Prüfung auf Gleichheit (Integer)		
Actual	^CHAR	Istwert
Expected	^CHAR	Sollwert

assertStringNotEqual		
Prüfung auf Gleichheit (Integer)		
Actual	^CHAR	Istwert
Expected	^CHAR	Sollwert

assertNStringEqual		
Prüfung auf Gleichheit (Integer)		
Actual	^CHAR	Istwert
Expected	^CHAR	Sollwert
size	UDINT	maximale Länge

assertNStringNotEqual		
Prüfung auf Gleichheit (Integer)		
Actual	^CHAR	Istwert
Expected	^CHAR	Sollwert
size	UDINT	maximale Länge

assertREALEqual		
Prüfung auf Gleichheit (Gleitkomma)		
Actual	REAL	Istwert
Expected	REAL	Sollwert
Granularity	REAL	Genauigkeit

assertREALNotEqual		
Prüfung auf Gleichheit (Gleitkomma)		
Actual	REAL	Istwert
Expected	REAL	Sollwert
Granularity	REAL	Genauigkeit

6.2.3.7. Datentypen

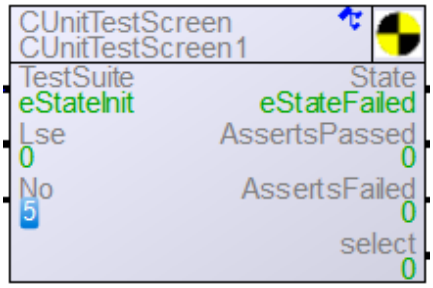
EState_t	
Status	
eStateIdle	
eStatePrepare	
eStateExecute	
eStatePassed	
eStateFailed	
eStateFailedFatal	
eStateError	

6.2.4. CUnitTestScreen

6.2.4.1. Beschreibung

Diese Klasse dient als Basisklasse für den Test von HMI-Funktionen.
Der Anwender muss eine konkrete Testklasse davon ableiten und kann diese dann bei der Testsuite registrieren.
Die Instanz bezieht sich dabei immer auf eine bestimmte Seite der Visualisierung.

6.2.4.2. Übersicht



6.2.4.3. Elemente

Name	Beschreibung
m_Screen	Schnittstelle zur Visualisierung

6.2.4.4. Schnittstelle

Client	Beschreibung
TestSuite	Verbindung zur Testsuite-Instanz
Lse	Verbindung zum Lasal Screen RTK
No	Nummer der Seite

Server	Beschreibung
State	Aktueller Status des Tests
AssertsPassed	Anzahl erfolgreicher Asserts
AssertsFailed	Anzahle fehlerhafter Asserts
Select	Auswahl

6.2.4.5. Methoden

prepare
siehe CUnitTestBase

execute
siehe CUnitTestBase

cleanup
siehe CUnitTestBase

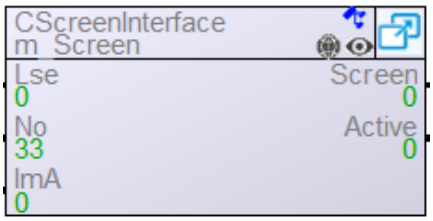
open	
Die Funktion dient zum Öffnen der Seite	
bRetcode	true = Befehl erfolgreich false = Befehl fehlerhaft

isActive	
Die Funktion dient dazu den Status der Seite zu lesen	
bRetcode	true = Seite geöffnet false = Seite nicht geöffnet

6.2.5. CScreenInterface

Die Klasse dient als Schnittstelle zu den Seiten der Visualisierung.
Eine Instanz davon ist Element der Klasse CUnitTestScreen.
Als Basisklasse dient die Klasse _Screen aus der Systembibliothek.

6.2.5.1. Übersicht



6.2.5.2. Schnittstelle

Client	Beschreibung
Lse	Verbindung zum Lasal Screen RTK
No	Nummer der Seite
ImA	Typ *IMA_PICTURE* *IMA_WINDOW*

Server	Beschreibung
Screen	
Active	1 = Seite Aktiv 0 = Seite nicht Aktiv

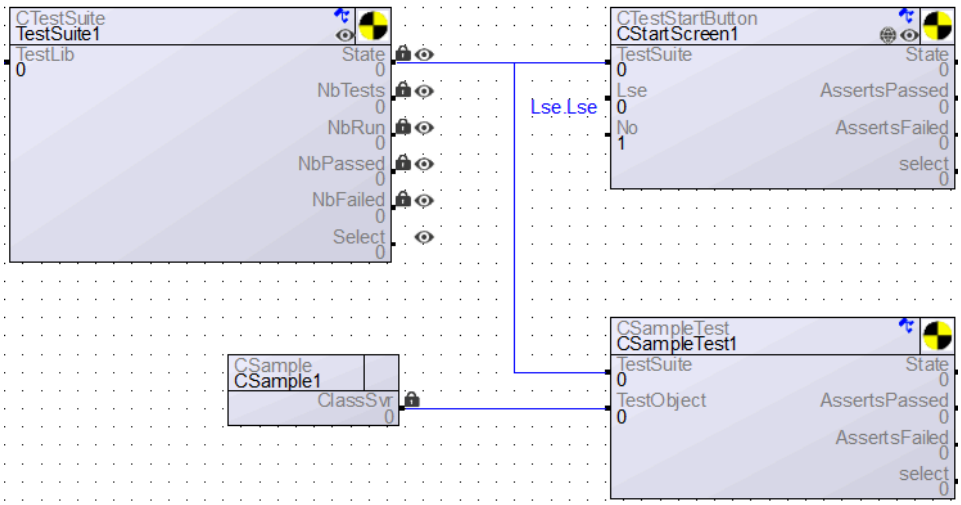
6.2.5.3. Methoden

S_Start	
Die Funktion wird aufgerufen wenn die Seite geladen wird	

S_End	
Die Funktion wird aufgerufen wenn die Seite verlassen wird	

6.3. Beschreibung des Beispiels

Im Beispiel sind drei Klassen enthalten die die Anwendung für Ablauftest und Visualisierungstest darstellen sollen.



6.3.1. Anwendungstest

Die Klasse **CSample** soll eine beliebige Anwendung darstellen die gestestet werden soll. Die darin enthaltene Methode Addiere(dValue1, dValue2) gibt laut Spezifikation ebenfalls wieder einen DINT Wert zurück.

Die zugehörige Testklasse heisst üblicherweise wie die zu testende Klasse mit dem Postfix *Test*. Hier also **CSampleTest**.
Für das Beispiel benötigen wir kein *prepare* oder *cleanup*. Daher wurde nur die *execute*-Methode überladen.

6.3.2. Visualisierungstest

Für den Testfall nehmen wir an das die Spezifikation einer Visualisierung besagt das jede Seite einen Softkey haben muss mit dem der Anwender auf die Startseite zurückkommt (Home-Button). Dieser Homebutton muss jeweils auf dem ersten Softkey von links sein.

Eine Instanz der Klasse **CTestStartButton** bildet jeweils für eine Seite des Visualisierungsprojekts den gewünschten Testfall ab.
Bei Aufruf von Prepare wird eine bestimmte Seite (Konfiguration in *No*) geöffnet (erster Test)
Bei Aufruf von Execute wird versucht über den ersten Softkey wieder zurück zur Startseite zu wechseln (zweiter Test).
Wenn beide erfolgreich waren gilt der Test als bestanden (*PASSED*).